

RECAST

面向大型工业软件的
智能检索与自适应开发平台

项目说明书

RECAST 项目团队

2026 年 9 月

目录

第一章 项目概述

1.1 项目简介

RECAST 是一套面向大型工业软件的代码检索与开发辅助平台。系统解析代码仓库，建立符号索引和依赖关系，根据开发人员或编程智能体提出的需求，查找相关实现、接口和配置，并返回对应源码、文件位置及版本信息。

大型工程中的一项功能通常分散在多个文件中。编程智能体即使找到入口函数，也可能需要继续查找调用方、数据类型和配置项。RECAST 预先完成可以重复使用的源码解析和索引工作，再按任务读取相关内容，减少智能体反复搜索和阅读文件的次数。

项目面向百万至千万行、多仓库和多语言工程开展研发，支持功能定位、代码修改、软件迁移和系统维护。这里的规模与语言范围是建设和评测目标，当前实现范围及已有实验结果见第八章。

1.2 项目定位

RECAST 为现有 AI Coding 工具提供代码查询服务。系统负责保存源码版本、提取声明、分析依赖和维护索引；Coding Agent 根据查询结果制定修改方案、生成代码并执行验证。

接入仓库后，系统分别保存模块、文件、类型和函数信息。收到查询时，先结合关键词、符号名称和语义相似度查找候选，再补充理解这些代码所需的接口、类型和配置。返回内容按照模型输入长度限制进行筛选，并标明哪些依赖尚未解析、哪些内容因长度限制而省略。

因此，开发人员不仅能看到匹配的代码，还能检查它属于哪个版本、与哪些实现相连，以及现有信息是否足以支持修改。

1.3 核心应用场景

RECAST 主要面向大型制造、能源、交通、金融及央国企的软件研发部门，也适用于承担遗留系统改造的软件集成商和维护多仓库产品的平台团队。主要场景包括：

- (1) 根据自然语言需求，在大型仓库中定位相关模块、文件、类型和函数。
- (2) 查找多套业务系统中的重复功能，为系统精简和功能合并提供参考。
- (3) 在语言、框架或平台升级前查找可复用实现，并识别迁移所需的依赖。
- (4) 根据需求、接口、代码片段或错误日志检索企业内部已有代码。
- (5) 修改公共接口或基础模块前，查找跨仓库、跨模块的调用方。
- (6) 在 Java、ArkTS、Kotlin/KMP、C/C++ 等混合工程中分析跨语言调用和注册关系。
- (7) 向 Coding Agent 返回相关源码及其位置、版本和依赖，供其继续分析与修改。

1.4 项目建设目标

项目需要同时解决大规模建库、任务定位和依赖补充三个问题。索引必须能够随源码变化更新，查询必须在限定时间内找到相关实现，返回内容还应包含修改所需的接口和配置。具体目标如下：

- (1) 在百万至千万行工程上建立可增量维护的代码索引，测量建库时间、内存与更新开销。
- (2) 支持 Java、ArkTS、Kotlin Multiplatform、C/C++ 等语言，分别验证语法解析、语言内名称解析和跨语言关系。
- (3) 统一记录跨文件、跨模块和跨语言依赖，使每条关系可以追溯到源码或构建配置。
- (4) 根据自然语言需求查找代码，兼顾召回率、排序准确性和查询时延。
- (5) 在 token、文件数和源码行数限制内选择必需内容，减少重复片段。
- (6) 通过 MCP、HTTP 和 IDE 接口向不同 Coding Agent 提供查询功能，并检验其对后续修改的实际帮助。

1.5 项目总体能力

系统管理仓库、分支和分析版本，识别源码、构建文件、接口声明、配置及测试。解析结果按模块、文件、类型和函数组织，全文索引负责词法匹配，向量索引用于查找语义相近的内容，依赖关系用于补充相关实现。

查询可以包含自然语言、代码片段、符号名称、接口签名、异常日志或文件路径。系统合并不同索引的候选结果，再按照工程范围、语言和接口要求筛选。需要进一步分析时，通过依赖查询补充调用方、被调用函数、类型或配置。

最终返回的数据包括入选源码、来源版本、文件位置和依赖说明。下游 Agent 可以直接查看这些内容，也可以针对未解析或省略的部分继续查询。

1.6 项目当前阶段

目前已有原型能够完成仓库接入、源码分析、模块划分、任务检索和结果展示。当前代码还接入了受限并行解析、模块分析结果恢复、部分跨语言绑定，以及多文件生成后的固定行为测试。

这些实现已通过固定样例和功能测试，但还不能据此认定完整 SDK 语义、千万行规模或未知开发任务的效果已经达标。现阶段重点是补齐跨语言分析的适用场景，检查大仓中的建库和查询开销，并完善前端与翻译后端的操作流程。

后续将在固定硬件、数据库、模型和任务集上测量检索准确率、响应时间、依赖覆盖率，以及代码修改后的行为通过率。已有结果和待开展实验分别列于第八章。

第二章 背景与需求分析

2.1 行业背景与国家战略需求

2.1.1 国产软件与操作系统生态发展

国产操作系统和企业数字化系统的发展带来了平台适配、系统整合和技术栈升级需求。企业已有业务代码往往分散在不同部门、仓库、分支和技术栈中，研发人员需要先确认某项功能是否已经实现，再判断现有接口和运行环境能否满足新的需求。

重新开发全部功能会重复已有工作，直接复制旧代码又可能遗漏依赖。软件迁移因此需要先理解原有实现，再确定可以复用的部分和必须修改的部分。

2.1.2 AI Coding 技术发展趋势

AI Coding 已从代码补全和单文件生成扩展到多文件修改、错误修复和软件迁移，但代码生成能力并不能代替工程中的功能定位和依赖分析。

大模型无法在一次输入中读取百万行乃至千万行源码。按固定长度切块容易切断函数及其关联定义，逐文件搜索又会增加模型调用和读取时间。因此，需要预先建立可查询的代码索引，再按任务提供相关源码。

2.1.3 超大规模软件工程智能化需求

大型工业软件通常包含较多模块、技术栈和历史版本。一项修改可能涉及不同文件甚至不同语言。LSP 和字符串检索适合导航到已知符号，但反射、依赖注入、运行时注册及配置驱动的调用还需要其他分析方法。

向量相似度可以找到功能描述接近的代码，却不能说明它们是否使用相同接口、平台或依赖。要把检索结果用于修改和迁移，还需核对这些工程条件。

2.2 命题企业与业务背景

2.2.1 命题企业相关业务背景

本项目面向华为提出的“AI Coding 超大工程代码理解与快速索引”相关产业需求。研发人员需要在百万至千万行工程中，为新增、修改、删除、迁移和维护任务确定实现位置及相关依赖。

HarmonyOS 等软件生态中的多语言、跨模块和跨平台特征增加了分析难度。查询系统既要处理代码规模，也要识别上层接口与底层实现之间的连接方式。

2.2.2 大型软件工程与代码迁移场景

一个业务功能可能同时涉及入口、Service、Repository、公共数据结构、配置、Native 模块和测试。在 Java、ArkTS、Kotlin/KMP、C/C++ 混合工程中，它们还可能通过 JNI、N-API、接口声明或构建配置连接。

因此，开始生成目标代码之前，需要明确原功能由哪些文件共同实现、调用哪些接口、依赖哪些配置，以及哪些行为必须保留。只翻译入口函数容易遗漏这些条件。

2.3 命题需求分析

2.3.1 命题总体目标

赛题要求支持 Java、ArkTS、Kotlin Multiplatform、C/C++ 等语言的代码理解与检索。代码检索准确率目标不低于 90%，平均检索时间控制在 10 秒以内；对接 AI Coding 后，转换或生成代码的功能完备度目标不低于 90%。

这些数值需要在统一测试集和运行环境下验证，不能以少量演示用例通过代替正式验收。

2.3.2 命题指标拆解

百万至千万行规模要求索引能够在受控资源内构建和更新。多语言支持要求分别检查声明解析、名称解析和跨语言依赖。检索准确率要求自然语言需求能够定位到有效实现，时延要求则限制候选搜索、源码读取和结果整理的总开销。

功能完备度还要求将检索与后续修改一起验证：即使排名靠前的函数与需求相关，遗漏类型、配置或异常处理仍可能导致生成代码行为错误。

2.3.3 命题本质分析

给定一个自然语言开发任务，系统需要从大型工程中找到相关实现，补充修改所需的依赖，再在模型输入上限内整理这些内容。

这一过程对应四项工作：建立并维护代码索引，分析调用与绑定关系，查找与需求相关的实现，以及筛选供 Agent 阅读的源码。四项工作需要使用相同的工程范围和源码版本。

2.4 典型业务场景

2.4.1 功能定位场景

开发人员可能只知道需求，不知道函数名或文件位置。例如，“查找登录失败后转换为 HTTP 401 的处理逻辑”可能对应异常处理器、鉴权模块或网关组件，代码名称未必包含完整需求表述。

系统需要结合描述、符号和调用关系找到实现，并给出可以直接查看的源码位置。

2.4.2 代码迁移场景

类型系统、异常处理、并发模型、数值精度、内存管理和框架约定的差异，可能使迁移后的代码能够编译却无法保持原有行为。

迁移时需要核对接口、调用关系、数据结构、配置和测试，并记录代码来源、修改范围及验证结果，便于检查遗漏和恢复原文件。

2.4.3 大型工程修改场景

修改公共接口或基础类型时，影响范围可能跨越多个模块或语言。只读取目标文件容易遗漏调用方、实现类和配置，读取整个工程又会增加模型输入。

系统需要从被修改对象出发查找相关使用者，返回可能受影响的源码，并说明分析尚未覆盖的关系。

2.5 现有 AI Coding 工作方式分析

通用 Coding Agent 通常通过多轮搜索和读取逐步了解陌生工程：先查找关键词，读取命中文件，发现新的符号后继续搜索，再根据获得的信息制定修改方案。

这种方式便于根据新发现调整方向，但仓库规模和语言种类增加后，搜索与读取次数也会上升。上下文过少会遗漏关键类型和配置，过多则会包含重复或无关内容。

预先维护代码索引可以减少重复扫描；根据依赖关系补充源码可以减少部分人工探索。两者仍需通过任务测试检查是否减少了读取次数并保留了必要信息。

2.6 现有技术方案及其局限

2.6.1 关键词与全文检索

关键词、字符串和正则检索适合查找已知文件名、函数名、错误码或固定字符串，结果也容易核查。但用户表述与标识符不一致时，词法匹配可能漏掉相关实现。

2.6.2 LSP 与符号检索

LSP 和编译器符号索引可以查找定义、引用与类型，通常需要先确定一个已知符号。跨语言桥接、运行时注册和配置驱动的关系还受到语言服务及构建环境限制，不能假定单一 LSP 可以完整分析。

2.6.3 向量语义检索

向量检索将需求和代码说明编码为向量，能够在命名不同的情况下找到功能相近的实现。但相似代码可能属于不同调用路径、平台或模块，仍需检查接口和依赖是否适用。

2.6.4 传统 RAG

普通代码 RAG 常将源码按固定长度切块，建立向量索引后返回排名靠前的片段。固定切块可能截断函数或类型，也难以同时满足局部修复与模块迁移所需的不同范围。

2.6.5 Coding Agent 多轮探索

Agent 能够根据新发现继续搜索和读取文件，但搜索方向取决于当前已获得的信息。在大型仓库中，这可能带来重复读取或遗漏依赖，需要记录调用次数、耗时和最终任务结果来判断效果。

2.7 核心技术难点

2.7.1 超大规模代码仓建模

在可接受的时间和内存内解析大型工程，保存可局部查询的索引，并在源码变化后减少重复处理。

2.7.2 跨语言跨模块依赖分析

根据声明、注册项和构建配置识别关系，保存对应位置；无法确定的目标保留未解析状态，避免错误连接。

2.7.3 高精度低延迟代码检索

结合词法、符号、语义和模块信息选择候选，在召回相关实现的同时限制搜索范围、读取量和查询时间。

2.7.4 模型输入中的源码选择

从候选中选出当前任务必需的实现、接口、配置、类型和测试，合并重复范围，并核对最终文本的 token 数。

第三章 市场现状与竞品分析

3.1 市场供给现状

相关产品主要包括代码搜索与导航平台、规则化重构与迁移平台、特定技术栈升级工具，以及通用 Coding Agent。它们分别侧重大仓查询、批量代码变换、固定升级流程和自然语言交互。

RECAST 的研发重点是根据任务查找实现、补充依赖并整理模型输入。与现有产品比较时，需要分别检查索引规模、可分析的关系、查询效果和修改结果，不能仅比较功能名称。

3.2 主要产品对比

产品	主要能力	本项目需要比较的内容
Sourcegraph	支持多仓库关键词、正则、符号、提交和差异搜索；SCIP 索引用于定义与引用导航，精确索引不可用时可使用搜索式导航	比较索引规模、查询时延和关系查询；另测自然语言需求能否定位有效实现，以及是否返回所需依赖
GitHub Copilot	结合仓库内容回答问题和执行编码任务，并接入代码托管与开发协作流程	比较任务所需的搜索与读取次数、上下文中重复内容、来源标注及最终修改结果
Moderne / OpenRewrite	OpenRewrite 使用 Lossless Semantic Tree 和确定性 recipe 执行升级、修复与重构；Moderne 支持多仓库执行和管理	对已知变换规则的任务比较正确率与批量处理效率；对需先查找参考实现的任务，比较定位与适配效果

产品	主要能力	本项目需要比较的内容
Amazon Q Developer	为受支持的 Java 和依赖版本升级提供自动化流程，结合构建、差异查看和人工接受完成迁移	在共同支持的升级任务上比较结果；多语言工程另行说明适用范围，避免将不同任务的指标直接对比

3.3 竞争判断

Sourcegraph 主要提供大规模代码搜索与导航，Moderne 和 OpenRewrite 擅长按确定规则执行批量变换，Amazon Q Developer 提供部分技术栈升级流程，通用 Coding Agent 则能根据自然语言灵活调用工具。

RECAST 尝试将大型工程索引、跨语言依赖分析和按任务选择源码结合起来。是否优于已有产品，需要在共同支持的任务和相同条件下测量；目前不能仅凭设计方案认定这些组合已经形成效果优势。

3.4 项目差异化方向

首先，检索需要找到承担目标功能的实现，即使命名与用户需求不同。其次，排序时检查调用关系、类型、构建条件和目标接口，减少语义相近但无法用于当前工程的结果。

检索结果还应包含必要依赖、来源版本和文件位置，并说明尚未解析或因长度限制而省略的内容。这样，开发人员可以核查结果，Agent 也可以继续查询缺少的信息。

系统与 Agent 分别承担可重复的索引处理和具体任务判断。解析、关系记录及基础查询由系统执行，模型在较小的候选范围内分析适用性并生成代码。

3.5 竞品对标方法

对比采用相同的脱敏多语言仓库、需求查询、硬件和超时策略。检索报告 Recall@K、Precision@K、MRR、nDCG、平均时延和 P95；依赖分析按关系类型报告准确率、召回率及跨模块路径完整度。

上下文评价记录必需代码覆盖率、重复内容比例、token 数和后续任务成功率。对不开放底层索引接口的产品，只比较能够直接观察和复现的结果，并注明产品版本与测试日期。

第四章 面向大型工程理解的系统框架

4.1 从功能描述到实现位置的检索问题

大型工程中的一次功能修改通常始于自然语言需求，最终却需要落实到具体的函数、类型、配置和调用关系。两者之间存在明显的表示差异：需求描述的是业务行为，代码按照语言语法和工程结构组织，而完整行为又可能分散在多个文件甚至不同语言中。例如，“调整上传文件的大小限制”既涉及入口方法，也可能涉及流式读取过程中的累计检查、异常类型和上层错误处理。只返回名称中包含 `upload` 或 `size` 的片段，不能充分说明修改应当发生在哪里。

这种差异使代码检索面临两个相互关联的问题。首先，用户往往不知道目标符号的准确名称，单纯的关键词匹配容易遗漏已有实现；其次，即使找到了语义相近的函数，局部正文也未必包含理解和修改该函数所需的依赖。前者要求系统建立从业务职责到代码实体的映射，后者要求系统在命中之后继续恢复实现之间的联系。因此，检索结果除相关源码外，还应列出代码所在的文件和版本、调用的接口，以及暂时无法确定的依赖。开发者据此判断代码是否适用于当前任务，并决定还需查看哪些文件。

工程规模增大后，查找源码和补充依赖所需的时间、内存及模型输入也会增加。若每个任务都重新扫描工程并让模型阅读大量文件，相同结构将被反复解析，模型输入和查询时延也会随工程增长。将全部源码提前切块并向量化虽然能够减少重复扫描，但固定切块会割裂函数和依赖，且高相似度片段可能集中于同一个实现。为减少这部分开销，系统预先解析工程并保存索引；收到查询后，只读取与任务有关的模块、类或函数。这样，每次查询无需重新处理整个仓库。

RECAST 的处理过程分为四步：按源码版本建立索引，根据需求定位模块和符号，补充相关调用与类型定义，最后在模型输入上限内筛选并整理源码。前两步减少重复解析和无关搜索，后两步补足修改所需信息并去除重复内容，供 Agent 制定修改方案和执行验证。

4.2 离线建模与在线查询的分层架构

系统将处理过程划分为离线建模与在线查询两条路径。离线处理保存源码快照，提取文件、符号和依赖记录，再生成模块树、全文索引和向量索引。在线查询读取已发布的索引版本，查找候选代码，补充直接相关的依赖，最后将源码及其位置、版本整理为模型输入，本文称为“上下文”。解析器和数据库负责源码提取与存取，模型主要判断模块职责和候选代码是否符合需求。

层次	核心对象与处理	向下游提供的能力
工程接入	仓库标识、项目边界、源码快照、内容哈希	确定一次分析的输入范围与版本
结构与职责建模	声明提取、关系绑定、模块提案、层级校验	从文件与符号映射到功能职责
索引与版本存储	结构记录、模块结果、全文与向量索引	按版本和范围执行局部查询
任务检索	多种索引召回、候选融合、工程约束、依赖扩展	将自然语言需求定位到源码片段
上下文与开发协同	去重、预算控制、未解析与省略说明、Agent 接入	为修改、适配和验证提供可追溯输入

其中，模块树与依赖图承担不同作用。模块树表达功能范围的包含关系，适合回答“哪个子系统负责这项功能”；依赖图表达调用、引用与绑定关系，适合回答“修改这个入口还涉及哪些实现”。向量索引中的近邻图只负责加速相似度搜索。三者通过模块或符号标识关联。模块归属、代码依赖和向量相似性分别记录，不能用“语义相近”代替“存在调用关系”。

离线与在线分离之后，关键问题转为如何防止分析结果与源码错配。系统为每次读取固定仓库、分析版本和项目范围，并要求符号、模块与源码片段服从同一版本约束。新版本构建期间，已经开始的查询继续读取原来的索引版本；新索引校验并发布后，后续请求再读取新版本。这样，后台更新不会使一次查询前半段读取旧声明、后半段读取新实现。

4.3 技术机制之间的协同关系

代码理解不能只依靠更大的模型输入。结构解析能够稳定提取声明和位置，却难以直接判断业务职责；模型能够解释职责，却可能给出重叠、遗漏或缺乏源码依据的模块划分。本项目因此以结构记录约束模型提案，由模型解释边界、宿主验证归属，再将通过校验的结果写入模块树。模块树的每一次细化都对应可检查的文件集合，而不是仅增加一层自然语言摘要。

有了模块树，检索可以从职责范围缩小到类和函数，但上层判断仍可能有偏差。如果查询只能沿一个模块分支向下搜索，错误路由会使正确实现从候选中消失。因此，模块内检索和全局符号检索共同提供候选：模块摘要提供范围线索，声明和正文检索保留直接命中路径，融合阶段再处理重复及粒度差异。

合并各检索通道的候选结果后，系统还需查找理解这些代码所必需的依赖。扩展不能无限沿图传播，否则公共工具、配置和调用方会迅速占满上下文。系统据此引入任务相关性、关系类型和资源上限，将扩展限制在解释主要实现所需的局部子图；随后合并重叠的源码范围，并按照输入长度限制筛选内容。最终结果包含相关实现、调用关系和源码位置，同时列出未能解析或因长度限制而省略的内容。

第五章 依赖约束的分层建模与增量索引

5.1 多语言源码的统一结构表示

多语言工程的首要困难，是不同语言前端产生的对象无法直接比较。Java 的类与方法、TypeScript 的导出声明、C/C++ 的函数和注册入口，在语法节点、名称作用域和构建方式上均有差异。如果检索侧直接依赖各语言的原始 AST，就需要为每一种语言重写查询逻辑，也难以把跨语言关系放到同一条路径中。

系统因此将语言前端的解析结果归一化为文件、符号和依赖三类记录。文件保存语言、角色、内容哈希和解析状态；符号保存声明种类、限定名称、签名及源码范围；依赖保存源端、目标端、关系类型、解析状态和对应的源码位置。语法前端负责发现“声明了什么”，后续名称解析或编译器分析负责确定引用所指向的声明，两者共同写入统一模型，查询接口不直接接触解析器内部节点。

符号标识不能仅由行号生成。插入注释或调整排版会改变位置，却未必改变声明本身。现有实现结合文件内 AST 声明标识、限定名称、签名和语言构造符号键，再把行列范围作为版本内的位置属性。源码快照提供实际正文，文件与内容哈希用于核对输入。由此，声明标识、版本位置和实际文本可以分别维护，减少格式变化对关联结果的干扰。

语法解析也不能代替编译语义。重载、条件编译和不同依赖版本可能让同一名称对应多个目标，而缺失 SDK 或构建配置又可能使目标无法确定。系统为关系分别记录已解析、有歧义或未解析状态：只有能够确定源声明和目标声明的关系才建立连接；其余关系记录候选目标或未解析原因，供后续查询和人工核查。目标语言支持因而按“结构可解析、语言内可绑定、跨语言可追踪”逐级建设，而非仅凭文件扩展名宣称完整支持。

5.2 大仓解析中的峰值内存与流水线背压

大型源码建库的资源压力不仅来自输入文件。解析树、符号表、依赖候选、模型输入和待写入记录可能同时驻留内存；当数据库写入速度低于解析速度时，尚未落库的结果会继续累积。即使每个文件都能独立解析，无界的阶段队列仍可能使整体内存失控。因此，优化目标应从“单个解析任务更快”扩展到“各阶段以受控速率协同推进”。

本项目将建库拆为扫描快照、结构解析、关系构建、结果写入与检索索引几个阶段。文件读取与解析采用受限批次，解析工作进程按批次回收，结果再分批写入。当前流水线已经限制了解析资源和写入批量；进一步的背压设计同时约束待处理文件数与字节量，当下游队列达到上限时暂停上游生产，避免大量小文件和少量大文件共享一个不合适的计数阈值。

设阶段 i 的并发数为 p_i ，单个工作单元的平均服务速率为 μ_i ，则稳定流水线的吞吐受最慢阶段限制：

$$X \leq \min_i (p_i \mu_i). \quad (5.1)$$

增加解析并发只在解析是瓶颈时有效；若写入已饱和，继续增加进程主要扩大队列和内存。峰值内存可进一步分解为：

$$M_{\text{peak}} \approx M_{\text{facts}} + p_{\text{parse}} M_{\text{worker}} + M_{\text{queue}} + M_{\text{write}}. \quad (5.2)$$

其中，四项分别表示驻留结构记录、解析工作进程、阶段队列和写入缓冲的开销。该表达式用于识别资源来源，不能以某个阶段的采样峰值代替整个系统峰值。

解析资源被限制后，结构写入和检索索引生成可能成为新的瓶颈。源码正文、声明信息和向量记录的更新频率与读取方式不同，故应分别保存并按需读取；写入端按批次提交，检索索引生成阶段仅处理需要编码的表示。后续分片与可恢复的任务调度以缩小 M_{facts} 和故障重算范围为目标，而不是假定增加线程便能线性提高千万行建库性能。

5.3 从目录边界到功能职责的自适应分层

目录是源码的物理组织方式，功能模块则反映代码共同承担的职责，两者并不总是一致。一个目录可能同时包含多个业务能力，同一能力也可能跨越接口层、服务层和持久化层。直接以目录作为最终模块容易造成职责混杂；反过来，让模型一次读取整个仓库，又会使输入规模和输出校验成本随工程增长。

系统以目录、包、文件角色和已解析依赖生成有限候选组，再由 Agent 判断候选之间的职责关系。每个候选提供文件规模、代表路径、公开接口与源码片段编号，组

间依赖汇总成边界线索；模型仅在必要时读取少量代表源码。这样，模型处理的是能够解释的局部范围，宿主仍保有完整的文件集合，可独立核对提案是否遗漏或越界。

对父范围 m 的一次拆分，设文件集合为 $F(m)$ ，子范围为 m_1 至 m_k 。有效拆分要求至少产生两个子范围、子集合不相交，并保持父范围的文件归属完整：

$$k \geq 2, \quad \bigcup_{i=1}^k F(m_i) = F(m), \quad F(m_i) \cap F(m_j) = \emptyset \ (i \neq j). \quad (5.3)$$

父节点不重复直接拥有子节点的文件，而是聚合其文件、类型和方法统计。无法解析或源码信息不足的文件通过未归属记录说明；校验失败的提案不能直接发布。上述约束把自然语言划分转换为可执行的集合检查，使模型解释与工程完整性相互配合。

划分深度由职责、规模和剩余预算共同决定。若当前范围已经具有单一职责，继续拆分只会增加导航层次和摘要成本，应当停止；若存在可区分的接口与实现边界，则继续细化。输入不足、模型失败或预算耗尽时，节点进入待细化状态，保留现有粗粒度范围及停止原因。小工程可以停在浅层，大工程的不同分支也可以具有不同深度，因而不需要为所有项目强制生成固定六层结构。

自适应划分的调度还受到父子依赖约束。子任务的输入范围只有在父提案通过校验后才能确定，而同一父节点下的独立分支可以并行分析。设计中的调度单位因此是“版本、父范围、候选集合与分析配置”共同标识的任务，解析、模型和向量计算分别使用受限资源池。完成一个分支即可保存其结果，不必等待整棵树结束后统一落盘。

为使并行任务能够恢复，持久记录需要保存输入内容哈希、运行状态、尝试编号和输出哈希。工作进程领取任务后获得本次执行编号，超时重试会生成新的编号；写入结果时同时核对输入版本与执行编号，迟到结果不能覆盖新尝试。父摘要在所需子结果满足条件后聚合，未完成分支继续显式保留。因此，重试可以复用已完成的分支，同时拒绝旧任务迟到的写入，防止同一棵模块树混入不同轮次的分析结果。

5.4 构建上下文与跨语言绑定恢复

统一记录解决了“如何表示关系”，但跨语言关系仍需要找到实际连接点。Java 到 Native 的调用可能经过 JNI 命名约定或动态注册，ArkTS 到 C/C++ 的调用可能经过 N-API 导出，KMP 则通过公共声明与平台实现连接。同名匹配在这些场景下并不充分：导出名、函数签名、注册范围和参与构建的目标必须共同成立，才能形成可信路径。

跨语言分析先识别上层调用入口，再根据导出名、签名和注册项查找底层实现，并记录每一步对应的源码位置。以注册式 Native 调用为例，先从上层声明确定方法名和签名，再查找注册表中对应项，最后解析该项引用的底层函数。图中分别保留声

明到注册项、注册项到实现的连接及对应的源码位置。若注册名相同但签名或库范围不同，则保留为不同候选，不能用较高文本相似度消除真实歧义。

构建上下文用于限定候选适用范围。Java 需要考虑依赖类路径和目标版本，C/C++ 需要考虑编译单元、宏与头文件搜索路径，KMP 需要区分公共源集和平台源集。分析结果因此应绑定构建配置哈希；配置变化即使没有修改函数正文，也可能改变关系目标。仅按语法匹配的关系与经过编译器确认的关系分别标记，查询时可以据此判断是否还需核对构建配置。

图查询围绕任务入口提取有限路径：先经过直接调用或绑定边到达实现，再按任务需要查看类型、配置与调用方。跨语言路径可以跨文件和模块，但其每一段都必须保留来源。性能评价同时检查依赖边准确率和关键路径覆盖，因为若一条迁移所需路径中间缺少注册边，即使其余多数边正确，也不足以支撑完整功能追踪。

5.5 基于变化类型的增量失效与版本发布

源码持续更新后，全量重建会重复解析未变化文件，而仅更新改动文件又可能留下失效关联。例如，某个公开方法被删除，调用方正文虽然未变，旧依赖却已不再有效；构建依赖版本变化也可能影响多个符号的绑定。因此，增量更新需要区分“输入是否变化”和“派生结论是否仍成立”。

系统首先比较内容哈希，确定变化文件集合，并复用兼容的解析结果。随后根据变化类型扩展失效范围：方法正文变化主要影响实现片段与所属模块说明，公开签名变化还影响引用该声明的关系，构建配置或注册信息变化则影响依赖其条件的绑定。模块摘要沿包含关系向必要祖先传播，向量缓存依据实际编码文本与模型名称和版本复用。文件路径相同并不足以证明向量可以复用。

设初始变化集合为 A_0 ，依赖这些文件或符号生成的记录集合为 $\text{Derived}(A)$ ，需要更新的模块祖先为 $\text{Ancestors}(A)$ ，则增量分析反复执行以下扩展，直至不再产生新的失效对象：

$$A_{t+1} = A_t \cup \text{Derived}(A_t) \cup \text{Ancestors}(A_t). \quad (5.4)$$

实现时根据源文件、符号与生成记录之间的反向映射定位受影响的关系，并按正文、接口和构建变化限制传播规则，避免把每次局部修改都扩展成整仓重建。无法安全界定的变化应扩大重算范围，以正确性优先于缓存命中率。

重建结果写入新的分析版本，结构、模块与检索索引分别记录进度。只有必需结果通过完整性和可查询性检查，才切换对外可见版本；失败时保留上一有效版本及失败原因。正在执行的查询继续持有其原有版本，旧结果在不再被读取后回收。增量正确性通过相同最终源码下的全量与增量结果对照验证，再统计重算文件比例、发布时延和资源收益。

第六章 任务驱动的分层检索与上下文整理

6.1 检索粒度与模块范围选择

检索粒度决定返回单个函数、类还是整个模块。函数适合定位局部逻辑，类或接口适合查看类型定义和方法约定，模块与子系统则适合解释职责及较大修改范围。若所有查询只返回固定大小代码块，模块迁移会缺少范围，而局部修复又可能收到过量材料。因此，系统将用户选择的主要结果粒度，与后续补充源码和依赖的粒度分开处理。

请求首先固定仓库、分析版本和工程范围，再选择自动、函数、类、模块或子系统模式。显式粒度具有优先权；当前范围没有相应结果时返回不可用状态，而不是悄悄换成其他类型。自动模式依据可用索引组织多粒度候选，必要的接口、配置与依赖仍可跨粒度补充。粒度选择只改变查询与结果组织，不触发重新解析。

分层检索先利用上层模块的职责摘要定位相关分支，再在这些范围内检索类和函数。其收益来自减少细粒度比较范围，但错误上层判断也可能放大漏检。因此，设计中保留一条不依赖模块路由的全局声明与正文召回通道，与模块范围内的候选共同进入融合。模块树提供方向，全局通道提供直接命中的补充，两者的配额和效果通过任务集调节。

当一项需求涉及多处修改时，查询不能只在单个模块中搜索。例如，“修改上传限制并调整异常返回”同时涉及校验与错误处理，如果排名靠前的候选全是大小校验函数，异常返回逻辑就可能被遗漏。后续将分别识别需求中的校验和错误处理要求，选择各自对应的实现，并保留两者的调用关系。

6.2 声明、正文和模块摘要的混合检索

自然语言和代码标识符具有不同匹配特征。“上传大小限制”可以与语义相近的说明匹配，但具体的 `parseRequest`、`FileUploadBase` 或异常名更依赖词法与结构信息。单一向量召回容易忽略精确标识符，单一全文召回又难以跨越不同命名表达。因此，

系统对符号声明、源码正文和模块摘要构建多种检索视图，并在每种视图内结合全文与向量通道。

不同通道的原始分值无法直接比较：全文相关度受到词频与长度影响，向量相似度受到模型与编码方式影响。融合阶段使用候选在各通道中的名次形成稳定贡献。对候选 d ，其融合得分为：

$$\text{RRF}(d) = \sum_{j: d \in L_j} \frac{w_j}{k_0 + \text{rank}_j(d)}. \quad (6.1)$$

其中， L_j 为第 j 个通道的结果列表，名次从 1 开始， w_j 为通道权重， k_0 为平滑常数；未命中的通道不贡献分值。当前存储实现使用以 60 为平滑常数的排名融合，任务服务再分别限制声明、片段和摘要候选。

合并后的候选还需对应到具体函数、类或模块。源码片段命中可能只覆盖函数中部，服务根据片段所属的声明映射到函数或类，避免把某段分支误当成完整 API；多种索引命中同一符号时，合并该符号的检索结果并记录各通道的命中情况，而不是重复占据结果位置。模块结果则关联其已发布范围和代表符号，以便从模块检索结果继续读取具体实现。

候选集合形成后再执行较昂贵的判断。工程、语言和目标接口约束先排除明确不适用的结果，然后对有限候选比较任务覆盖、接口兼容性和边界依赖。这样，精排处理的是已经经过结构筛选的集合，而不是整仓源码。当前任务查询流程主要采用排名和规则选择，后续将加入模型对任务适用性的判断，并在相同候选数量限制下比较排序效果，并单独记录模型调用增加的延迟。

6.3 在线读取放大与时延控制

向量搜索本身并非总是在线查询的主要成本。若系统在筛选之前加载大量文档正文，或为了补充一个符号而读取完整结构索引，数据库传输和对象构建就可能超过近邻搜索耗时。随着工程增长，检索时间会更多地消耗在“没有进入最终上下文的数据”上，形成逻辑上的读取放大。

系统先取得数量受限的候选标识，筛选后再读取对应正文。工程范围在查询阶段过滤，符号、片段与摘要通道分别设置上限；候选融合后，再按标识读取符号和源码范围。任务检索依赖局部符号、邻接关系和源码接口，不通过整仓列表拼出一次结果。由此，在线读取量主要受候选与扩展预算约束，而不是直接等于仓库规模。

为观察无效读取，定义一轮查询的读取放大率：

$$A_{\text{read}} = \frac{\text{Bytes}_{\text{read}}}{\text{Bytes}_{\text{delivered}}}. \quad (6.2)$$

其中，分子为查询实际读取的源码字节数，分母为最终返回的源码字节数；没有

返回源码的请求单独报告。当前统计不包含数据库磁盘读取量和网络协议开销。该指标不是越接近 1 越好，因为合理的候选筛选需要读取未入选内容，但持续偏高往往提示过滤过晚或重复取数。结合候选命中率与必需代码覆盖率，才能判断减少读取是否牺牲了检索效果。

在串行阶段边界下，查询时延可分解为：

$$T_{\text{query}} = T_{\text{encode}} + T_{\text{recall}} + T_{\text{rank}} + T_{\text{expand}} + T_{\text{compile}}. \quad (6.3)$$

各项分别对应编码、召回、排序、扩展和上下文整理；一个阶段内存在并发时，以该阶段实际墙钟耗时计量，避免重复累加重叠时间。各阶段共享请求截止时间，独立召回可以在受限并发下执行，已计算的查询向量在通道间复用；依赖扩展接近预算时停止，并报告尚未覆盖的范围。耗时改善应来自缩小不必要的工作量和重复计算，而不是在结果中隐藏超时或将未完成请求计为成功。

6.4 相关依赖的查找与需求覆盖

首次检索得到的函数可以作为查找依赖的起点，但其正文未必包含修改所需的全部信息。函数可能使用外部类型、调用内部工具或依赖构建配置，修改接口还可能影响调用方。若沿全部关系展开，公共依赖又会把局部任务扩展到大量无关源码。因此，图扩展需要同时考虑任务、关系方向和资源成本。

系统先读取主要实现，再查询与它直接相连的部分依赖。函数内部修改优先补充被调用实现和类型，公共接口调整需要关注调用方与实现类，跨语言迁移则优先追踪注册绑定和平台适配路径。每次补充内容时记录它与主要实现或任务要求的关系，并保留未解析关系；已访问符号按版本与标识去重，防止环和高连接节点导致重复探索。

为减少只返回同类代码的问题，方案将任务中的不同要求组成集合 U ，并为每项要求设置权重 $w(u)$ 。对候选代码片段 e ， $\text{Cov}(e)$ 表示它涉及的需求项；已选片段集合 C 的覆盖得分定义为：

$$F(C) = \sum_{u \in \bigcup_{e \in C} \text{Cov}(e)} w(u). \quad (6.4)$$

需求项可以对应入口实现、目标接口、关键分支、必要依赖或测试。判断某段代码是否涉及一项需求，需要查看其声明、调用关系和具体逻辑；模型生成的说明只能作为分析线索，不能证明代码行为已经符合要求。

这一表示解释了为什么 Top-K 相似度不能替代上下文选择。两个高度相似的函数可能支持同一项需求，加入第二个函数的新增得分很小；一个相似度稍低的异常定义却可能补上尚未覆盖的错误处理要求。选择过程因此需要比较新增覆盖与输入 token 数，优先补充当前结果尚未包含的接口、分支或依赖，而不是只按相似分数继续追加。

6.5 输入长度限制下的源码筛选与整理

上下文选择同时受到 token、文件数和源码行数约束。若只对源码正文估算 token，最终添加标题、版本、关系和省略说明后仍可能超限；若优先堆入所有元数据，又可能使真正需要修改的实现无法进入窗口。因此，长度限制必须作用于最终返回文本，并在来源说明与实际源码之间保留合理空间。

上下文整理模块（代码中的 Context Compiler）先按仓库、版本、路径和内容哈希识别重复片段，再合并相互包含的源码范围。客户端可以提交已收到片段的标识和内容哈希，避免后续请求重复返回。候选加入前尝试渲染完整 Markdown，检查 token、唯一文件数和源码行数；同时为裁剪说明预留空间。最终用 cl100k_base 对实际导出文本重新计量，保存实际用量及未能包含的内容。

选择源码时，在需求覆盖得分中扣除重复内容的惩罚：

$$\max_{C \subseteq E} [F(C) - \lambda \text{Redundancy}(C)]. \quad (6.5)$$

$$\text{Tokens}(C) \leq B, \quad \text{Files}(C) \leq F_{\max}, \quad \text{Lines}(C) \leq L_{\max}. \quad (6.6)$$

E 为候选代码片段集合， B 为输入预算， $F(C)$ 为上一节的覆盖得分， $\text{Redundancy}(C)$ 衡量重复源码范围，控制冗余惩罚强度。需求覆盖的重复已在集合并集中去除，冗余项进一步约束文本层面的重复。由于每段代码的输入长度和需求覆盖相互影响，直接枚举所有子集需要指数级搜索，在线查询应使用受限候选上的启发式选择。

在线查询采用贪心近似：每轮优先选择单位 token 带来最多新增需求覆盖、且重复较少的片段。实现先以候选独立序列化的增量 token 估计排序成本，接纳时再对完整文本精确计量，避免为每个候选反复编码全部组合。记 $J(C)$ 为覆盖得分减去冗余惩罚，则候选的增量评分为：

$$\text{Score}(e | C) = \frac{J(C \cup \{e\}) - J(C)}{\max(1, \Delta \text{Tokens}(e | C))}. \quad (6.7)$$

每轮挑选仍满足长度约束且新增得分较高的代码片段，合并被包含范围，并在新增内容后重新评估受影响候选。任务明确指定的入口或接口优先保留；长度限制不足以容纳这些内容时，明确列出未返回的部分。规模较小的候选集可以用精确枚举作为离线对照，评价启发式与最优值的差距，而不把启发式输出称为全局最优。

整理后的结果包含源码版本、主要实现、相关依赖，以及未能提供的内容及其原因。partial 表示存在未解析、裁剪或其他限制，已返回的内容仍可使用；Agent 根据未解析或省略说明继续查询符号、依赖或源码。补取继续遵守固定版本和已返回片段去重，形成“检索—补齐—修改—验证”的迭代过程。是否提供了足够信息，需要对照人工标注的必需源码和后续修改结果判断；满足输入长度限制并不意味着信息完整。

第七章 Agent 协同开发与工程实现

7.1 HTTP、MCP 与 IDE 的检索接口

离线索引和在线算法只有进入实际开发流程，才能减少 Agent 重复搜索与阅读代码的成本。不同客户端若各自拼接检索结果，就容易在版本选择、源码范围和省略原因的解释上产生差异。因此，系统使用统一的 ContextPacket 数据结构返回源码、版本、依赖和省略说明。HTTP、MCP 和 IDE 均调用同一检索服务和上下文整理模块。

HTTP 通过 POST /v1/task-search 接收任务请求，返回结构化上下文；MCP 通过 search_task_context 返回适合 Agent 阅读的内容，并提供 get_symbol、get_dependencies 和 read_source_excerpt 等局部工具。一次请求携带明确范围，服务返回对应的版本与来源。Agent 先查看首次检索结果，再根据未解析或省略的内容补查符号、依赖或源码，避免把整个仓库交给模型反复筛选。

浏览器工作台和 VS Code 扩展将这些协议映射为需求输入、工程范围、检索粒度和源码预览。页面保留用户能够理解的任务选择，预算由宿主默认值和服务约束承担。模块树按需分页读取，模块细化状态、解析失败原因和源码声明分别展示；选择模块只改变任务范围和详情，不改变文件中真实的类与方法列表。

系统以不同状态表达建库进度：结构索引可用意味着能够定位声明，模块结果可用意味着已有职责分析，检索索引可用意味着这些结果已进入召回。检索索引生成失败时可以复用已校验提案重试，模型提案失败则保留原因和上一有效结果。状态分离使用户能够判断哪一步需要处理，也为自动恢复提供明确入口。

7.2 参考检索、适配规划与多文件生成

复用迁移进一步要求回答“参考实现如何适合目标工程”。语义相似只能说明功能接近，不能证明接口、类型和外部依赖能够直接复用。例如，参考代码使用同步返回值，而目标接口要求异步执行，逐句翻译即使语法正确，也可能违背目标调用方式。因此，迁移在检索之后增加显式规划，根据参考代码的行为和目标工程的接口要求制定修改步骤。

现有工作区翻译后端接收 Context 与 Spec，由 Analyzer 整理来源到目标的名称

映射、依赖处理策略和步骤依赖，再由 Translator 执行多文件修改。每个步骤声明所需文件和前置步骤，目标读写范围由宿主限定。模型只能在已经提供的上下文和允许读取的文件中补充源码和依赖，每个生成步骤保留所参考代码的路径和版本。

多文件修改还需要处理运行中断与用户后续编辑。服务保存修改前内容、预期修改后内容和应用状态，并在写入前持久记录。恢复执行时重新核对文件基线，回退时先检查全部待恢复文件，确认没有被后续修改覆盖，再执行逆向恢复。单个工作区限制并发写入，使计划状态、文件状态与恢复记录保持一致。

编译反馈将计划与生成连接成可执行循环。服务运行宿主配置的编译命令，取得诊断后交给模型修复；完成任务前再次核对通过编译的源码快照是否仍是当前内容。这样能够避免“某一轮编译通过，随后又修改文件却直接结束”的情况。编译结果只覆盖相应语言和工程约束，行为保持还需要下一节的独立验证。

7.3 从编译成功到行为验收

编译器能发现语法和类型问题，却无法证明上传限制、异常映射或数据处理结果符合需求。若用编译成功作为迁移完成标志，错误实现仍可能通过流程。因此，系统分别记录编译检查、行为测试和代码差异审阅结果，完成状态应说明实际通过了哪些检查。

行为验证方案由宿主配置命令和测试范围，模型不能自行替换验收标准。测试用例围绕 Spec 中的正常路径、边界条件和异常行为组织；对迁移任务，可在相同输入下比较参考实现与目标实现的输出及可观察副作用。测试失败时保留失败用例、输出和对应源码版本，将反馈交给后续修复。若只有编译记录，结果继续标记为 compilation-only。

为使行为验证可以审阅，每次运行需要关联任务、计划哈希、目标基线、文件差异和验证命令，并对测试执行前后的源码内容进行核对。只有全部必需检查通过且检查后未再修改对应文件，才进入待审阅状态；用户审阅后再回填目标工程。通过这些记录，可以从最终改动查回使用的参考代码、修改计划和测试结果。

端到端功能测试按固定任务执行上述步骤：登记工程并发布索引，检索参考实现，生成预算内上下文，形成适配计划，执行多文件修改，运行编译与行为测试，最后核对差异和恢复能力。该过程既检查单项技术可调用，也检查它们之间传递的范围、版本和状态是否一致。

第八章 实验设计与阶段结果

8.1 验证基线与实验问题

本章以 chiparon/weichai 的 9-8-v 分支、提交 c3f09d3 (2026 年 9 月 8 日 22:35:58, UTC+8) 作为现有原型基线。代码用于核对机制接通情况，仓库运行说明及交接材料用于整理既有实验记录。huawei 分支在此基础上实现新增机制，并合入 2026 年 9 月 9 日上游最新的 codex/guochuang-ui 分支 (a02e903)。本次以固定样例验证合并后的功能；没有重新执行大仓建库或远程模型实验。以下历史数字均注明所属基线，新实现的验证范围单独说明。

实验围绕四个问题组织：受限建库能否在真实大仓中控制资源；Agent 层级能否形成合法且有意义的职责范围；多粒度检索能否在合理时延内返回真实代码；上下文与开发协同能否提高后续修改的成功率。前三项已有部分工程记录，第四项及完整跨语言能力仍需要独立任务集验证。

技术环节	c3f09d3 已有基础	huawei 接入及后续验证边界
代码建库	源码快照、分批解析、版本存储、局部查询	并行解析与在途字节背压；资源分片及千万行验证待开展
职责建模	Agent 提案、自适应层级、归属校验与发布	同级分支并行、决策持久化与重试复用；多宿主调度待扩展
语言与关系	已接入语言的结构记录、统一关系接口	新增目标语言结构前端及保守绑定；完整 SDK 语义待验证
任务检索	多种索引的混合检索、粒度约束、有限依赖扩展	召回通道并行、查询向量复用、源码读取字节数与时延统计
上下文	源码读取、范围去重、最终文本计量	按需求词、代码用途及命中符号选择片段；是否包含必需内容待标注评测

技术环节	c3f09d3 已有基础	huawei 接入及后续验证边界
开发协同	多文件翻译、编译反馈、运行恢复	固定行为测试、源码快照核验、HTTP 查询与回滚；UI 与翻译后端仍需联调

基线默认支持 Java、TypeScript、JavaScript、Python、C#、Go 和 Rust。huawei 分支新增 C/C++、Kotlin 与 ArkTS 结构入口，其中 ArkTS 复用 TypeScript 兼容语法，ArkUI 专用语法保留解析诊断。跨语言样例验证 JNI 短导出名、明确 N-API 注册链及同模块 expect/actual 的连接；重载、重复实现或缺少注册的情况保留未解析或歧义。该结果证明有限规则下的路径可追踪，不能替代编译器、SDK 和实际构建配置验证。

8.2 百万行建库：解析受限后的写入瓶颈

既有容量实验采用真实 VS Code src, 源码提交为 c5f02db50896ef310d3c909628e651675644fcef, 共 7,708 个文件、2,356,695 物理行，包含源码、测试及其他角色，没有通过复制文件扩充规模。结构索引记录 287,471 个符号和 117,420 条语法依赖；解析成功、部分解析和不支持文件分别为 7,691、10 和 7 个。

阶段或资源	既有测量记录	统计范围
扫描解析	约 385 秒	旧扁平模块基线的源码建库
结构写入	约 869 秒	受本机数据库日志配额影响
检索索引	约 784 秒	与扫描解析分别计时
解析进程采样峰值	约 376 MiB	父进程与解析子进程合计
全流程主进程峰值	约 1.39 GiB	不含数据库和模型进程

这些结果说明，受限解析已能处理百万行真实输入，但建库时间并不主要集中于语法解析。结构写入与检索索引生成均长于扫描解析，继续提高解析并发未必缩短总

时间；应优先检查数据库批量写入、索引维护与日志配额影响。两个内存数字的统计范围不同，也不能据此推断整套系统只需要数百 MiB 内存。

已有增量基础支持复用兼容结构结果，并保留全量与增量等价测试。进一步实验将变化分为正文、公开签名、文件增删、构建配置和跨语言注册五类，对照相同最终源码的全量构建结果，检查旧边撤回、端点重绑定和模块摘要更新，再记录重算比例与发布耗时。这样，增量收益才建立在结果一致性的前提上。

8.3 Agent 分层：归属覆盖与细化质量的区别

9 月 8 日的 DeepSeek 大仓记录复用 VS Code 主工程 7,702 个文件的固定索引，生成 72 个层级节点、4 个根范围，其中包含 66 个模块和 6 个子系统。终端深度为 0 至 3，最多 4 层；14 个节点已拆分，1 个为叶节点，57 个仍待细化，文件归属覆盖为 7,702 / 7,702。

模型日志记录 24 次完成的层级请求，其中 16 个有效决策被用于生成模块树，节点中 15 个标记模型来源、57 个保留结构来源。请求数、决策数和节点数分别描述调用、校验与结果，不应混为同一个成功指标。模块建模约 73.842 秒，检索索引生成约 24.139 秒，未重新解析全部源码；完整遍历核对了父子文件、类型和方法聚合一致性。

文件覆盖完整说明每个文件均能在当前范围中找到归属，却不能证明 57 个待细化范围已经具有准确功能边界。小工程进一步说明了两类指标的差异：Java 目标工程 commons-fileupload-java-skeleton 生成 8 个浅层功能模块，覆盖 58 / 61 个文件，一个构建配置不支持解析，两个源码文件解析失败，均保留原因。该结果没有为了覆盖率把源码信息不足的文件强行分配，也没有为了固定深度增加子系统。

后续模块实验采用人工职责标注，分别计算归属完整性、重复归属、职责一致性和边界依赖，并统计模型调用、待细化比例与恢复重算量。并行版本与串行版本使用相同输入及配置比较结果约束和吞吐；中断恢复实验则在父提案发布、子任务运行和检索索引生成阶段注入故障，核对是否复用已完成结果、拒绝迟到写入并保留合法层级。

8.4 检索与上下文：源码返回结果与查询时延

旧大仓基线使用 64 维 hash 向量查询 CodeEditorWidget executeEdits editor text model，约 4.96 秒返回 17 条源码片段，最终文本为 7,813 / 8,000 token，包含 executeEdits 的完整方法。该记录验证了百万行源码中的有界读取与精确计量，但查询包含真实 API 标识符，不能据此推断一般自然语言任务的排序质量。

最新 Agent 模块结果接入检索后的三项记录如下。它们继续使用 hash 向量，任务中包含模块名称和 API，主要检查根据模型生成的模块树能否查到同一版本的实际

源码。

查询粒度	记录耗时	结果与上下文状态
子系统	7.917 秒	命中“会话”，6 个文件，partial
模块	15.537 秒	命中“CLI 入口”，6 个文件，partial
自动	8.231 秒	返回“会话”等结果，5 个文件，partial

三项请求均返回实际源码，但模块查询超过 10 秒目标，说明高层结果的代表源码选择与关联读取仍需分析。正式评估应按编码、召回、排序、依赖扩展和上下文整理分阶段记录耗时，同时统计读取数据量，判断延迟来自数据库访问、候选管理还是上下文整理。三个不同请求不构成完整时延分布，不能替代均值、P95 或超时率。

独立中文语义回归使用 Commons FileUpload Java 源码、TypeScript 工程和 384 维 multilingual E5，既有 HTTP 10/10、MCP 6/6 用例通过。这里的通过表示对应源码及协议断言成立，并非大规模任务准确率达到 100%。语义模型结果与 hash 容量基线分开报告，后续以统一中文任务集标注入口、关键分支和必要依赖，分别计算检索质量、必需代码覆盖率、冗余和补取次数。

8.5 代码迁移验证与消融实验

翻译交接材料记录了真实模型完成 Java 到 TypeScript 的两文件翻译，并在 10 轮调用后编译通过；当时没有行为测试结果，也没有完整页面操作流程证明。该记录支持“多文件生成与编译反馈可以运行”，尚不能支持功能完备度达标。完整实验需在同一任务上继续执行行为用例，并核对测试对应的源码快照与最终写入结果。

对照组采用关键词搜索、单纯向量召回、多种索引的混合检索和同时补充依赖并按需求筛选源码的完整方案，固定模型、任务、候选预算和时间上限。消融实验分别移除模块摘要、依赖扩展、覆盖选择或范围去重，观察缺少哪一环会增加漏检、重复输入或人工修复。检索指标与开发指标共同记录，避免优化相似度名次却没有改善实际修改。

评价环节	主要指标	与技术机制的对应关系
建库与更新	总耗时、峰值资源、重算比例、发布时延	检验流水线 and 增量失效的收益
模块与关系	职责边界、依赖边准确率、路径覆盖、未解析率	检验层级及跨语言连接是否可信
检索	Recall@K、MRR、nDCG、均值与 P95 时延	检验融合和路由的质量与成本
上下文	必需代码覆盖率、token、重复范围、补取次数	检验源码选择是否完整且重复较少
开发结果	编译与行为通过率、人工修复量、恢复成功率	检验端到端任务是否完成

命题要求的检索准确率至少 90%、平均检索时间 10 秒以内和代码功能完备度超过 90%，需要在明确口径下分别验收。检索准确率应先定义任务级成功条件和相关性标签，功能完备度应对应需求行为用例；现有少量接口记录与编译记录不足以证明三项指标同时达标。千万行规模实验采用真实工程，单独记录硬件、模型、数据库、并发与冷热缓存条件，使结果能够复现和比较。

8.6 新增实现验证与后续实验

新增机制以固定样例验证输入、输出和失败路径：并行解析受到工作进程数与在途字节的双重限制；同级模块决策乱序返回后仍按稳定顺序组装；重建协调器后可以复用已校验决策；超时模型回复不会写入有效检查点。跨语言注册变化后重新构建关系，并将增量结果与相同最终源码的全量结果比较，检查旧连接是否被清除。

开发流程样例从真实 Java 源文件建库与检索开始，将固定版本的检索结果交给受控工具调用，生成两个目标文件，再运行语法检查与行为断言。另一个样例的初始实现能通过语法检查，却遗漏负数处理；行为测试拒绝完成，修复并重新验证后才记录 behavior-verified。写入后旧结果失效，验收文件禁止修改，差异记录支持重启后回滚。

这些样例使用确定性模型脚本，检验服务协议和完整执行流程，不代表远程模型在未知任务上的成功率。后续仍需完成结构记录与检索索引分片、按变化类型复用模块结果和完整 SDK 语义验证，再以真实大仓及开发任务集分别评价资源成本和修改质量，检查新增机制能否在实际任务中降低开销并提高修改正确率。

第九章 项目团队与组织分工

9.1 团队总体情况

RECAST 团队成员来自计算机科学与技术、数据科学、人工智能和公共管理等专业，分别承担算法研发、系统开发、实验测试和项目材料整理工作。

指导教师负责审查技术路线、分析企业需求和评议阶段结果；学生团队负责实现、测试及演示。各项任务按模块分工，并通过接口联调和代码评审检查集成结果。

9.2 团队组织与任务分工

研发工作分为源码解析与索引、依赖分析、检索排序、后端接口、前端展示和实验评估。各模块明确输入、输出和测试要求，接口发生变化时同步修改调用方和相关文档。

9.3 指导教师

指导教师结合软件工程、程序分析和工业智能体研究，参与技术方案讨论与实验设计。以下资历信息沿用原计划书。

姓名与职务	研究经历及项目分工
杨振，助理教授	山东大学计算机科学与技术学院助理教授，研究方向包括软件工程、程序分析与智能化软件开发。近 5 年在 FSE、ASE、ISSTA、TSE、TOSEM 等期刊和会议发表论文 30 余篇，其中第一作者或通信作者 CCF-A 类论文 8 篇、CCF-B 类论文 12 篇。入选 2025 年度山东省青年科技人才托举工程、山东省引进海外科技人才。在本项目中指导代码索引、跨语言依赖分析和检索技术方案。

姓名与职务	研究经历及项目分工
孙景昊，教授	山东大学计算机学院教授，研究方向包括工业智能体、自动驾驶运行时软件技术及实时调度与优化理论。主持国家自然科学基金重大研究计划培育项目、面上项目、青年项目、省自然科学基金项目和华为胡杨林基金等项目。近 5 年以第一作者或通信作者在 IEEE Transactions on Computers、IEEE Transactions on CAD、ACM Transactions on Embedded Computing Systems、RTSS、RTAS、EMSOFT、DAC 等期刊和会议发表论文 18 篇，其中 TOP 类期刊论文 6 篇，CCF-A/B 类论文多篇。在本项目中指导企业需求分析、应用场景设计和产品规划。

表 9.1 项目指导教师信息

9.3.1 学生团队

姓名	专业方向	工作分工
吕曦冉	数据科学	项目负责人，协调进度和技术方案，参与需求分析与架构设计，组织模块集成和成果材料。
唐程	计算机科学	负责代码索引和工程数据处理，参与解析流程、结构表示及大规模源码组织。
赵旭东	计算机科学	负责程序结构和依赖分析，参与模块关系识别及跨模块调用信息整理。
张汇智	智能科学与技术	负责语义匹配与检索算法，参与需求分析、向量检索和候选排序。
赵瀚兴	计算机科学	负责后端服务和接口设计，参与模块集成、部署及运行环境配置。
王润鲁	计算机科学	负责功能实现和模块集成，参与接口联调及稳定性问题排查。
张博文	计算机科学	负责检索流程优化与实验评估，参与候选召回、排序设计和结果分析。
彭煜程	数据科学	负责实验数据处理、测试方案和结果分析，参与性能测量及验证流程实现。

姓名	专业方向	工作分工
马丽	公共管理	负责需求与文档整理，参与应用场景分析、项目材料准备和演示组织。
吴东贤	公共管理	负责资料整理、测试支持和应用场景调研，协助准备演示与成果总结。

表 9.2 学生团队成员信息

9.4 师生协同与项目组织

指导教师参与关键技术讨论、实验设计和阶段评审，学生团队完成需求调研、算法实现、系统开发与功能测试。讨论以具体问题为依据，例如索引耗时过长、依赖解析失败或测试结果不一致，并记录处理方案和复测结果。

团队按技术模块同步进展，通过代码评审、接口联调和实验分析检查实现是否符合方案。企业场景中的功能需求和使用反馈用于调整开发优先级；每项调整都应落实为具体改动和相应测试。

第十章 项目总结与后续规划

10.1 项目阶段性总结

10.2 当前成果总结

10.3 当前存在的不足

10.4 技术迭代规划

10.5 产品落地规划

10.6 规模化验证规划

10.7 企业应用与推广规划